Binary Search

1. Binary Search

Sorted integer array di hui hai (duplicates nahi). Humein ek `target` dhoondhna hai — agar mile to uska index return karna, warna `-1`.

Array sorted hai, isliye har step par beech ka element check karte hain:

- agar beech wala element == target → mil gaya, return index.
- agar target > beech → target right half mein hoga → left = mid+1.
- agar target < beech → target left half mein hoga → right = mid-1.
  Is process ko tab tak repeat karo jab tak `low <= high`. Agar cross ho gaye → nahi mila.

Iterative Code:

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    // Iterative binary search
    int binarySearch(const vector<int>& nums, int target) {
        int low = 0, high = (int)nums.size() - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (nums[mid] == target) return mid;
            else if (nums[mid] < target) low = mid + 1;
            else high = mid - 1;
        }
        return -1;
    }
};

int main() {
    vector<int> a = {3, 4, 6, 7, 9, 12, 16, 17};
    int target = 6;

    Solution s;
    int ind = s.binarySearch(a, target);

    if (ind == -1) cout << "The target is not present.\n";
```

```
    else cout << "The target is at index: " << ind << "\n";
    return 0;
}
```

Complexity Analysis

Time Complexity: In the algorithm, in every step, we are basically dividing the search space into 2 equal halves. This is actually equivalent to dividing the size of the array by 2, every time. After a certain number of divisions, the size will reduce to such an extent that we will not be able to divide that anymore and the process will stop. The number of total divisions will be equal to the time complexity. So the overall time complexity is O(logN), where N = size of the given array.

Space Complexity: 0(1), no extra space being used

Recursive Approach:

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    int binarySearchRec(const vector<int>& nums, int low, int high, int target) {
        if (low > high) return -1;
        int mid = low + (high - low) / 2;
        if (nums[mid] == target) return mid;
        if (nums[mid] < target) return binarySearchRec(nums, mid + 1, high, target);
        return binarySearchRec(nums, low, mid - 1, target);
    }
public:
    int search(const vector<int>& nums, int target) {
        return binarySearchRec(nums, 0, (int)nums.size() - 1, target);
    }
};

int main() {
    vector<int> a = {3, 4, 6, 7, 9, 12, 16, 17};
    int target = 6;

    Solution s;
    int ind = s.search(a, target);
```

```
    if (ind == -1) cout << "The target is not present.\n";
    else cout << "The target is at index: " << ind << "\n";
    return 0;
}
```

Complexity Analysis

Time Complexity: In the algorithm, in every step, we are basically dividing the search space into 2 equal halves. This is actually equivalent to dividing the size of the array by 2, every time. After a certain number of divisions, the size will reduce to such an extent that we will not be able to divide that anymore and the process will stop. The number of total divisions will be equal to the time complexity. So the overall time complexity is O(logN), where N = size of the given array.

Space Complexity: 0(1), no extra space being used

2. Implement Lower Bound

Sorted array diya hua hai. Humein **lower bound** dhoondhna hai →
pehla index jahan arr[i] >= x ho.
Agar aisa koi index na mile → return n (array size).

2) Lower Bound Example (simple)

- {1, 2, 2, 3}, x=2 → **index 1** (pehli baar 2 ya usse bada value mila)
- {3, 5, 8, 15, 19}, x=9 → **index 3** (15 ≥ 9)

Brute Force Approach

- Array ko left se right tak scan karo.
- Jispe pehli baar arr[i] >= x mile → wahi answer.
- Nahi mila → return n.

⏳ Time: O(N)
📦 Space: O(1)

```
#include <bits/stdc++.h>

using namespace std;


class LowerBoundFinder {

public:

    int lowerBound(const vector<int>& arr, int n, int x) {

        for (int i = 0; i < n; i++) {

            if (arr[i] >= x) return i;

        }

        return n;

    }

};


int main() {

    vector<int> arr = {3, 5, 8, 15, 19};

    int n = arr.size();

    int x = 9;
```

```
    LowerBoundFinder f;

    cout << "The lower bound is the index: " << f.lowerBound(arr, n, x) << "\n";

    return 0;

}
```

Optimal Approach

ahan sorted array ka fayda:
Binary search se sirf **arr[mid] >= x** condition check karna hai.

Cases:

1. **arr[mid] >= x**
   → mid answer ho sakta hai → ans = mid
   → left side me aur chhota valid index ho sakta hai → high = mid - 1

2. **arr[mid] < x**
   → mid chhota hai → bigger element right side me
   → low = mid + 1

Loop khatam → ans return.

⏳ Time: O(log N)
📦 Space: O(1)

```
#include <bits/stdc++.h>
using namespace std;

class LowerBoundFinder {
public:
    int lowerBound(const vector<int>& arr, int n, int x) {
        int low = 0, high = n - 1;
```

```
        int ans = n;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (arr[mid] >= x) {
                ans = mid;          // potential answer
                high = mid - 1;    // search left side
            } else {
                low = mid + 1;     // search right side
            }
        }
        return ans;
    }
};

int main() {
    vector<int> arr = {3, 5, 8, 15, 19};
    int n = arr.size();
    int x = 9;

    LowerBoundFinder f;
    cout << "The lower bound is the index: " << f.lowerBound(arr, n, x) << "\n";
    return 0;
}
```

3. Implement Upper Bound

Sorted array `arr` of size `N` and integer `x` diye hain. **Upper bound** = smallest index `i` such that `arr[i] > x`.
Agar aisa index na mile → return `N`.

Sorted hone ki wajah se first element **strictly greater** than x ko binary search se jaldi find kar sakte ho; otherwise scan linearly.

Brute Force

- Left se iterate karo.
- Pehli position jahan `arr[i] > x` ho → return `i`.
- Agar loop finish ho gaya → return N.

Why O(N)? worst-case jab sab elements ≤ x ho tab tumhe poora array traverse karna padega → N comparisons.
Space O(1): koi extra data structure ya recursion stack nahi.

```
#include <bits/stdc++.h>
using namespace std;

int upperBoundBrute(const vector<int>& arr, int x) {
    int n = (int)arr.size();
    for (int i = 0; i < n; ++i) {
        if (arr[i] > x) return i;
    }
    return n;
}

int main() {
    vector<int> arr = {1,2,2,3,3,5};
    int x = 2;
    cout << "Upper bound index (brute): " << upperBoundBrute(arr, x) << "\n"; // expected 3
    return 0;
}
```

Optimal (Binary Search)

Use binary search to maintain a candidate `ans` (initialized to n).
At each step compute `mid = low + (high - low)/2` (safer against overflow).

- If `arr[mid] > x`: `mid` is a potential answer → store `ans = mid`, move left (`high = mid - 1`) to find smaller index.
- Else (`arr[mid] <= x`): discard left part (`low = mid + 1`).

Loop ends when `low > high`. Return `ans`.

Why O(log N)? har step search space half ho jata hai → number of steps ~ $\log_2 N$.
Space O(1): iterative, no extra memory or recursion.

```
#include <bits/stdc++.h>
using namespace std;

int upperBound(const vector<int>& arr, int x) {
    int n = (int)arr.size();
    int low = 0, high = n - 1;
    int ans = n; // default if no element > x

    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] > x) {
            ans = mid;        // possible answer
            high = mid - 1;   // search left for smaller index
        } else {
            low = mid + 1;    // arr[mid] <= x, ignore left half
        }
    }
    return ans;
}

int main() {
    vector<int> arr = {3, 5, 8, 9, 15, 19};
    int x = 9;
```

```
    cout << "Upper bound index: " << upperBound(arr, x) << "\n"; // expected 4
    return 0;
}
```

Edge cases to mention

- Empty array → return `0` (because `n = 0`).
- All elements ≤ x → return n.
- All elements > x → return `0`.
- Duplicates around x (e.g., many elements equal to x) → upper bound returns first index **after** last x.

4. Search Insert Position

You have a **sorted array with distinct values** and a **target x**.
You must return the index where:

- target exists **OR**
- target should be inserted to keep array sorted.

This is the same as **lower bound**, because we need the first index where `arr[i] >= x`.

Because the array is sorted, the index where x should be inserted is:

👉 **the first element which is greater than or equal to x**
If no such element exists → x should be inserted at the end → return n.

This is exactly the same binary search we used for **lower_bound**.

We maintain:

- `low = 0`
  `high = n - 1`
- `ans = n` (default → insert at end if nothing ≥ x)

Steps:

1. Calculate `mid = low + (high - low) / 2`
2. Compare `arr[mid]` with x
    - If `arr[mid] >= x`
      → possible insertion index
      → update `ans = mid`
      → search left half (`high = mid - 1`)
    - Else
      → x is on right side
      → `low = mid + 1`
3. Continue until `low > high`
4. Return `ans`

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    int searchInsert(const vector<int>& arr, int x) {

        int n = arr.size();

        int low = 0, high = n - 1;

        int ans = n;   // default if x should be placed at the end


        while (low <= high) {

            int mid = low + (high - low) / 2;
```

```
            if (arr[mid] >= x) {
                ans = mid;      // potential insert position
                high = mid - 1; // search left side
            } else {
                low = mid + 1;  // search right side
            }
        }

        return ans;
    }
};

int main() {
    vector<int> arr = {1, 2, 4, 7};
    int x = 6;

    Solution s;
    cout << "The index is: " << s.searchInsert(arr, x) << "\n";
    return 0;
}
```

5. Floor and Ceil in Sorted Array

Given a **sorted array** `arr` of size n and a value x, find:

- **Floor(x):** largest value ≤ x
- **Ceil(x):** smallest value ≥ x

If floor or ceil does not exist → return **-1**.

Binary Search works perfectly because:

- **Floor** is the **rightmost** value ≤ x
- **Ceil** is the **leftmost** value ≥ x

So both can be found by slightly modifying binary search.

Floor Algorithm (Simple Explanation)

Goal → largest element ≤ x
Steps:

1. `low = 0`, `high = n-1`, `ans = -1`
2. Find `mid`
3. If `arr[mid] ≤ x`:
   → it is a valid floor → update `ans = arr[mid]`
   → but maybe a larger valid floor exists → go right (`low = mid + 1`)
4. Else (`arr[mid] > x`):
   → too big → go left (`high = mid - 1`)
5. Continue until `low > high`
6. Return `ans`

Ceil Algorithm (Simple Explanation)

Goal → smallest element ≥ x
Steps:

1. `low = 0`, `high = n-1`, `ans = -1`
2. Find `mid`

3. If `arr[mid] ≥ x`:
   → valid ceil → update `ans = arr[mid]`
   → try to find smaller ceil → go left (`high = mid - 1`)
4. Else (`arr[mid] < x`):
   → too small → go right (`low = mid + 1`)
5. Return `ans`

```
#include <bits/stdc++.h>
using namespace std;

class FloorCeilFinder {
public:
    int findFloor(const vector<int>& arr, int x) {
        int low = 0, high = arr.size() - 1;
        int ans = -1;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (arr[mid] <= x) {
                ans = arr[mid];     // potential floor
                low = mid + 1;      // try to find a larger floor
            } else {
                high = mid - 1;     // go left
            }
        }
        return ans;
    }

    int findCeil(const vector<int>& arr, int x) {
        int low = 0, high = arr.size() - 1;
        int ans = -1;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (arr[mid] >= x) {
                ans = arr[mid];     // potential ceil
                high = mid - 1;     // try to find smaller ceil
            } else {
                low = mid + 1;      // go right
            }
```

```
        }
        return ans;
    }

    pair<int, int> getFloorAndCeil(const vector<int>& arr, int x) {
        int floorVal = findFloor(arr, x);
        int ceilVal  = findCeil(arr, x);
        return {floorVal, ceilVal};
    }
};

int main() {
    vector<int> arr = {3, 4, 4, 7, 8, 10};
    int x = 5;

    FloorCeilFinder finder;
    auto ans = finder.getFloorAndCeil(arr, x);

    cout << "Floor and Ceil: " << ans.first << " " << ans.second << "\n";
    return 0;
}
```

Time Complexity: O(log N)

Because both floor and ceil use binary search:

- Search space halves every iteration
- Steps required ≈ $\log_2(N)$

Even two searches → O(log N) + O(log N) = **O(log N)**.

Space Complexity: O(1)

- Only a few variables: `low`, `high`, `mid`, `ans`
- No recursion, no extra arrays
- Constant memory usage.

Quick Dry Run (x = 5, arr = {3,4,4,7,8,10})

Floor

- 3 ≤ 5 → floor = 3 → go right
- 4 ≤ 5 → floor = 4 → go right
- 4 ≤ 5 → floor = 4 → go right
- 7 > 5 → go left
  → Final floor = **4**

Ceil

- 4 < 5 → right
- 4 < 5 → right
- 7 ≥ 5 → ceil = 7 → left
  → Final ceil = **7**

6. Last occurrence in a sorted array

You are given a **sorted array** of size N and a `target`.
You must return the **index of the last occurrence** of the target
(0-based indexing).
If target does NOT exist → return **-1**.

Because the array is sorted:

- The duplicates (if any) are grouped together.
- The **last occurrence** will be at the **rightmost** index where `arr[i] == target`.

Binary search can do this faster by:

- Finding `target`
- Then still searching **right side** for any later occurrence.

✅ Brute Force

Start from the **end** of the array and move backward:

- As soon as you find `v[i] == key`, return `i`.
- If not found → return `-1`.

Time Complexity → O(N)

Worst case: target not found → you scan all N elements.

Space Complexity → O(1)

```
#include <bits/stdc++.h>
using namespace std;

int lastOccurrenceBrute(int n, int key, const vector<int>& v) {
    for (int i = n - 1; i >= 0; i--) {
        if (v[i] == key) return i;
    }
    return -1;
}
```

✅ Optimized Binary Search

We maintain:

- `start = 0`
- `end = n - 1`
- `res = -1` (default if target not found)

During search:

1. Compute `mid`
2. If `v[mid] == key` →
    - store mid in `res`

    - move `start = mid + 1` (because last occurrence may be to the right)
3. If `key < v[mid]` → search left (`end = mid - 1`)
4. Else → search right (`start = mid + 1`)

Loop continues until `start > end`.

Why It Works?

Because even after finding the target, we **force binary search to continue rightwards** to capture the last occurrence.

Time Complexity → O(log N)

Binary search halves the search space every step.

Space Complexity → O(1)

Only variables used: start, end, mid, res.

```
#include <bits/stdc++.h>
using namespace std;

int lastOccurrence(int n, int key, const vector<int>& v) {
   int start = 0, end = n - 1;
   int res = -1;

   while (start <= end) {
      int mid = start + (end - start) / 2;

      if (v[mid] == key) {
         res = mid;        // store the index
         start = mid + 1;   // keep searching on the right side
      }
      else if       (key < v[mid]) {
         end = mid - 1;     // go left
      }
      else {
         start = mid + 1;   // go right
      }
   }
   return res;
```

```
}

int main() {
    vector<int> v = {3,4,13,13,13,20,40};
    int key = 13;
    int n = v.size();

    cout << lastOccurrence(n, key, v) << "\n";  // Output: 4
    return 0;
}
```

Quick Dry Run (Example 1)

Array: {3,4,13,13,13,20,40}, key = 13

start	end	mid	v[mid]	action
0	6	3	13	res=3, move right (start=4)
4	6	5	20	key < v[mid] → end=4
4	4	4	13	res=4, move right (start=5)
5	4	stop		

Final answer = **4**

For target = 13 → last occurrence index = 4
For target = 60 → -1 (not found)

7. Count Occurrences in Sorted Array

You have a **sorted array** of integers and a number **X**.
Your job is to find **how many times X occurs** in the array.

Because the array is sorted:

- All occurrences of X will be **together in a block**.
- If we find:
    - **first occurrence index**
    - **last occurrence index**

Then:

`count = last - first + 1`

Binary search is perfect for finding first and last position efficiently.

Brute Force Approach

Traverse the array and count how many times `arr[i] == X`.

✔ Time Complexity

O(N) → worst case check all elements.

✔ Space Complexity

O(1) → no extra memory.

```
int countBrute(const vector<int>& arr, int n, int x) {
    int cnt = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] == x) cnt++;
```

```
    }
    return cnt;
}
```

Optimal Approach — Using Binary Search

We use **two modified binary searches**:

1. **firstOccurrence()** – find first index where arr[mid] == x
2. **lastOccurrence()** – find last index where arr[mid] == x

Then compute:

```
count = last - first + 1
```

If `first == -1` → x not present → return `0`.

First Occurrence

- If arr[mid] == x → store mid, move **left** (high = mid - 1)
- Else normal binary search rules.

Last Occurrence

- If arr[mid] == x → store mid, move **right** (low = mid + 1)
- Else normal binary search rules.

This ensures we find the extreme ends of X's block.

```
#include <bits/stdc++.h>

using namespace std;


int firstOccurrence(const vector<int>& arr, int n, int x) {
    int low = 0, high = n - 1, first = -1;
```

```
    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == x) {
            first = mid;          // possible first occurrence
            high = mid - 1;       // go left to find earlier occurrence
        }
        else if (arr[mid] < x) {
            low = mid + 1;        // go right
        }
        else {
            high = mid - 1;       // go left
        }
    }
    return first;
}

int lastOccurrence(const vector<int>& arr, int n, int x) {
    int low = 0, high = n - 1, last = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
```

```
        if (arr[mid] == x) {
            last = mid;           // possible last occurrence
            low = mid + 1;        // go right to find later occurrence
        }
        else if (arr[mid] < x) {
            low = mid + 1;        // go right
        }
        else {
            high = mid - 1;       // go left
        }
    }
    return last;
}

int countOccurrences(const vector<int>& arr, int n, int x) {
    int first = firstOccurrence(arr, n, x);
    if (first == -1) return 0;  // x not present
    int last = lastOccurrence(arr, n, x);
    return last - first + 1;
}

int main() {
    vector<int> arr = {2, 2, 3, 3, 3, 3, 4};
    int x = 3;
```

```
    int n = arr.size();


    cout << "Total occurrences: " << countOccurrences(arr, n, x) << "\n";

    return 0;

}
```

Time Complexity → O(log N)

- first occurrence → O(log N)
- last occurrence → O(log N)
  Total → O(2 × logN) = O(logN)

Space Complexity → O(1)

Only a few variables, no extra storage.

8. Search Element in a Rotated Sorted Array

Given a sorted array of **distinct** integers that has been rotated at an unknown pivot, find the index of target k. If k is not present return -1.
(0-based indexing)

Even after rotation, **one** half (left or right of mid) is always sorted. Find which half is sorted, check if k lies inside that sorted half; if yes, search there, otherwise search the other half. Repeat (binary search style).



target = 8,
Considering the comparison made, such as target > arr[mid] (e.g., 8 > 2), we would expect the target to be in the right half. However, due to the array rotation, the number 8 is actually situated in the left half. This rotation creates a challenge in the elimination process.

Brute force

Scan array linearly, return `i` when `arr[i] == k`, else `-1`.

- Time: **O(N)**
- Space: **O(1)**

```
int linearSearch(const vector<int>& a, int k) {
    for (int i = 0; i < (int)a.size(); ++i)
        if (a[i] == k) return i;
    return -1;
}
```

Optimal approach (binary search on rotated array)

Either of one half is sorted

At each step:

- compute `mid`
- if `a[mid] == k` → return `mid`
- else check if left half `a[low]..a[mid]` is sorted (`a[low] <= a[mid]`)
    - if sorted and `a[low] <= k < a[mid]` → search left (`high = mid - 1`)
    - else → search right (`low = mid + 1`)

- otherwise right half is sorted
    - if `a[mid] < k <= a[high]` → search right (`low = mid + 1`)
    - else → search left (`high = mid - 1`)

Repeat until `low > high`.

This keeps O(log N) time and O(1) space.

```
#include <bits/stdc++.h>
using namespace std;

// Returns index of k in rotated sorted array a, or -1 if not found.
// Assumes all elements are distinct.
int searchInRotated(const vector<int>& a, int k) {
    int n = (int)a.size();
    int low = 0, high = n - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (a[mid] == k) return mid;

        // If left half [low..mid] is sorted
        if (a[low] <= a[mid]) {
            if (a[low] <= k && k < a[mid]) {
                high = mid - 1;  // k in left sorted half
            } else {
                low = mid + 1;   // k in right half
            }
        }
        else { // right half [mid..high] is sorted
            if (a[mid] < k && k <= a[high]) {
                low = mid + 1;   // k in right sorted half
            } else {
                high = mid - 1;  // k in left half
            }
        }
    }
    return -1; // not found
}

int main() {
    vector<int> arr = {7,8,9,1,2,3,4,5,6};
    int k = 1;
```

```
    int idx = searchInRotated(arr, k);
    if (idx == -1) cout << "Target is not present.\n";
    else cout << "The index is: " << idx << "\n"; // expected 3
    return 0;
}
```

9. Search Element in Rotated Sorted Array II

You are given a rotated sorted array (but now duplicates are allowed).
You must return **True** if element k exists, otherwise **False**.

Example:

```
[7, 8, 1, 2, 3, 3, 3, 4, 5, 6]
```

Sorted + rotated + duplicates.

Why this is harder than “Rotated Sorted Array I”? (Distinct Version)

In the distinct version:

👉 One half is **ALWAYS** sorted
(either left or right)

But **in duplicates version** this strict rule sometimes breaks.



Example:

```
[3, 3, 3, 3, 3]
```

Here:

```
arr[low] = arr[mid] = arr[high]   -> 3 = 3 = 3
```

Now, can you say left is sorted? right is sorted?
NO.
Both look identical.
The algorithm **cannot determine** the sorted half.

This is the **ONLY** reason why duplicates break the clean logic.

How do we fix the problem?

When you see:

```
arr[low] == arr[mid] == arr[high]
```

You cannot decide which half is sorted.

So the trick is:

👉 just shrink the array

```
low++
high--
```

This safely removes duplicate boundary noise
without affecting correctness.

After shrinking, one half will again become detectable.

This trick is the heart of the algorithm.

Full Intuition

At every step:

✔ Step 1: Check mid

If `arr[mid] == k` → found → return true

✔ Step 2: Handle the duplicate ambiguity

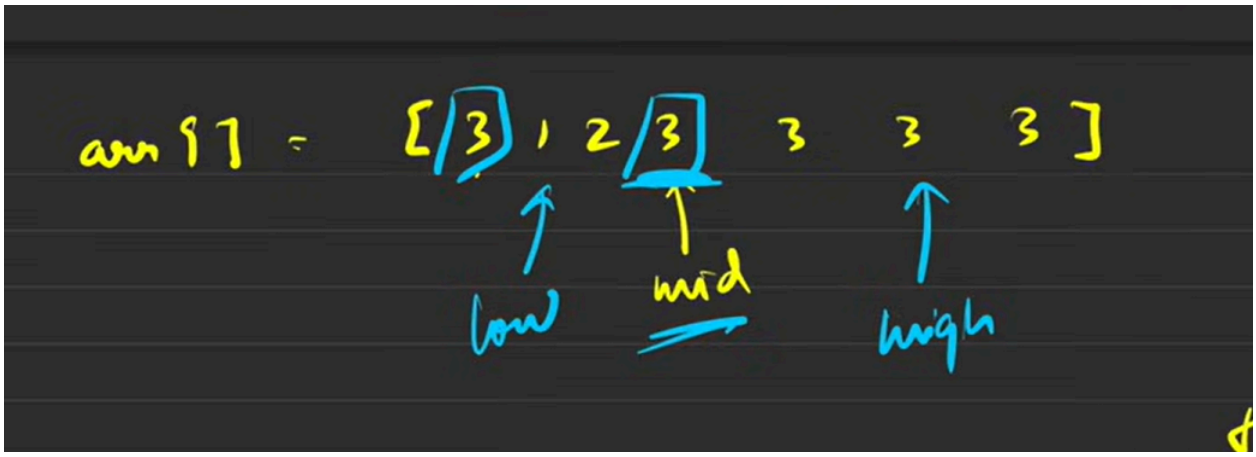
If

```
arr[low] == arr[mid] && arr[mid] == arr[high]
```

then:

👉 We cannot know which half is sorted
👉 So shrink:

```
low++
high--
continue
```



✔ Step 3: Identify which half is sorted (once duplicates are handled)

Case A: Left half sorted

```
arr[low] <= arr[mid]
```

Check if k lies in that left sorted range.

Case B: Right half sorted

Else → right half sorted
Check if k lies in that right sorted range.

```
#include <bits/stdc++.h>
using namespace std;

bool searchInRotatedArrayII(vector<int>& arr, int k) {
    int low = 0, high = arr.size() - 1;

    while (low <= high) {

        int mid = low + (high - low) / 2;

        // Case 1: found
        if (arr[mid] == k) return true;

        // Case 2: Cannot determine sorted half (duplicate trap)
        if (arr[low] == arr[mid] && arr[mid] == arr[high]) {
            low++;
            high--;
            continue;
        }

        // Case 3: Left half sorted
        if (arr[low] <= arr[mid]) {
            if (arr[low] <= k && k < arr[mid]) {
                high = mid - 1;  // target lies in left
            } else {
                low = mid + 1;   // target lies in right
            }
        }

        // Case 4: Right half sorted
        else {
            if (arr[mid] < k && k <= arr[high]) {
                low = mid + 1;   // target lies in right
```

```
            } else {
                high = mid - 1;  // target lies in left
            }
        }
    }

    return false;
}

int main() {
    vector<int> arr = {7, 8, 1, 2, 3, 3, 3, 4, 5, 6};
    int k = 3;
    cout << (searchInRotatedArrayII(arr, k) ? "True" : "False");
}
```

This problem CANNOT guarantee `O(log N)` for all cases.

✔ Best/Average: O(log N)

Because it behaves like normal binary search most of the time.

✔ Worst Case: O(N)

Why?

When many duplicates are present:

Example:

```
[3,3,3,3,3,3,3,3]
```

Every time:

```
arr[low] == arr[mid] == arr[high]
```

So binary search degenerates to linear shrinking:

```
low++
high--
```

Thus worst-case = **N/2 → O(N)**

10. Minimum in Rotated Sorted Array

Given a **sorted array (distinct values)** that is **rotated at an unknown pivot**, find the **minimum element**.

[4,5,6,7,0,1,2] → 0
[3,4,5,1,2] → 1

A rotated sorted array looks like this:

```
Original: [1,2,3,4,5]
Rotated : [4,5,1,2,3]
```

Notice:

👉 The array is split into **two sorted parts**.
👉 The **minimum element** is the **rotation point**.
👉 It is the **only element** that is smaller than its previous element.

This makes the problem perfect for **binary search**.

Brute Force

Traverse the entire array and keep track of the smallest element.

- Time: **O(N)**
- Space: **O(1)**

```
int findMinBrute(vector<int>& nums) {
    int ans = INT_MAX;
    for (int x : nums) ans = min(ans, x);
    return ans;
}
```

Optimal Approach — Binary Search

Even after rotation:

- **Right half is sorted** when `nums[mid] <= nums[high]`
- **Left half is sorted** when `nums[mid] >= nums[low]`

But we don't care which half is sorted.

We only care:

👉 **Is the minimum on the right or left?**

The Rule (Very Easy)

Compare **nums[mid]** with **nums[high]**:

Case 1: nums[mid] > nums[high]

This means mid is in the **left sorted part**,
so the minimum must be on the **right side**:

```
[4,5,6,7, 0,1,2]
       ↑       ↑
      mid    high
mid > high → go right
```

So:

```
low = mid + 1
```

Case 2: nums[mid] < nums[high]

This means the **right part is sorted**,
so the minimum is either at mid or in the left segment:

```
[4,5,6, 1,2]
        ↑ mid
nums[mid] < nums[high] → min is left side
```

So:

`high = mid`

✔ Loop ends when low == high

The index of minimum is found.

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
   int findMin(vector<int>& nums) {
      int low = 0, high = nums.size() - 1;

      while (low < high) {
         int mid = low + (high - low) / 2;

         if (nums[mid] > nums[high]) {
            // Minimum lies in the right half
            low = mid + 1;
         } else {
            // Minimum is in left half (including mid)
            high = mid;
         }
      }

      return nums[low]; // low == high
   }
};
```

Time Complexity: O(log N)

Binary search divides the array into half every step.

Space Complexity: O(1)

Only low, high, mid used.

11. Find out how many times the array has been rotated

Sorted array (distinct values) ko unknown pivot par rotate kiya gaya hai. Hume pata karna hai **kitni baar array rotate** hua — yani original sorted array ke mukable current array ka **smallest element ka index**. Woh index hi rotation count hota hai.

Examples:

- `[4,5,6,7,0,1,2,3]` → smallest `0` at index **4** → rotated 4 times
- `[3,4,5,1,2]` → smallest 1 at index **3** → rotated 3 times

(Assumption: distinct values — agar duplicates ho to algo thoda modify karna padega.)

Rotation cut point pe order **break** hota hai. Smallest element wahi hota hai jahan break hua. So find index of minimum.

Approach A — Brute force

Scan full array, track minimum value and its index.
Return index.

```
int findRotationsBrute(const vector<int>& a) {
    int n = a.size();
    int minIdx = 0;
    for (int i = 1; i < n; ++i)
        if (a[i] < a[minIdx]) minIdx = i;
    return minIdx;
}
```

Complexities: Time O(N), Space O(1).

Approach B — One-pass break-finding (slightly better idea)

Traverse once and find first i such that `a[i] > a[i+1]`.
Rotation count = `i+1`.
If no such i found → array not rotated → return 0.

```
int findRotationsOnePass(const vector<int>& a) {
    int n = a.size();
    for (int i = 0; i + 1 < n; ++i) {
        if (a[i] > a[i+1]) return i + 1;
    }
    return 0;
}
```

Complexities: Time O(N), Space O(1).
This is faster in practice than brute (you can stop early at the break).

Approach C — Optimal (Binary Search) — O(log N)

Key idea: Use same trick as *findMin in rotated sorted array*:

- Compare `a[mid]` with `a[high]`.
    - If `a[mid] > a[high]` → min is in right half → `low = mid + 1`.
    - Else → min is at `mid` or left → `high = mid`.
- Loop while `low < high`. When ends `low == high` → index of minimum → rotation count.

Why this works: In rotated sorted array the smaller half containing the minimum will be identified by `a[mid] <= a[high]`.

```
#include <bits/stdc++.h>
using namespace std;

int findRotationCount(const vector<int>& a) {
    int n = a.size();
    if (n == 0) return 0;          // edge: empty array (define as 0)
    int low = 0, high = n - 1;
```

```
    while (low < high) {
        int mid = low + (high - low) / 2;
        if (a[mid] > a[high]) {
            // min is to the right of mid
            low = mid + 1;
        } else {
            // min is at mid or to the left
            high = mid;
        }
    }
    // low == high is index of minimum element
    return low;
}

int main() {
    vector<int> arr = {4,5,6,7,0,1,2,3};
    cout << findRotationCount(arr) << "\n"; // prints 4
    return 0;
}
```

12. Search Single Element in a sorted array

You're given a **sorted** array where every element appears **exactly twice**, except **one element which appears only once**.
Find that unique element.

Examples:

```
[1,1,2,2,3,3,4,5,5,6,6] → 4
[1,1,3,5,5]             → 3
```

Brute Force – Approach 1 (Neighbor checking)

Because array is sorted, duplicates appear **side-by-side**:

1 1 | 2 2 | 3 3 | 4 | 5 5 | 6 6

The single element is the one
→ **not equal to left neighbor**
→ **not equal to right neighbor**.

✔ Algorithm

1. If array length = 1 → return arr[0].
2. Check first element (index 0):
   - If `arr[0] != arr[1]` → answer is arr[0].
3. Check last element:
   - If `arr[n-1] != arr[n-2]` → answer is arr[n-1].
4. For each middle index `i`:
   - If `arr[i] != arr[i-1]` AND `arr[i] != arr[i+1]` → return arr[i].

```
int singleNonDuplicate(vector<int>& arr) {
    int n = arr.size();
    if (n == 1) return arr[0];

    if (arr[0] != arr[1]) return arr[0];
    if (arr[n-1] != arr[n-2]) return arr[n-1];

    for (int i = 1; i < n-1; i++) {
        if (arr[i] != arr[i-1] && arr[i] != arr[i+1])
            return arr[i];
    }

    return -1; // guaranteed not to happen
}
```

⏱ **Complexity**

- **Time:** O(N)
- **Space:** O(1)

Brute Force – Approach 2 (XOR Trick)

💡 Why XOR works?

Properties:

- `a ^ a = 0`
- `a ^ 0 = a`

Pairs cancel out, only unique survives:

```
1^1 ^ 2^2 ^ 3^3 ^ 4 = 4
```

```
int singleNonDuplicate(vector<int>& arr) {
    int ans = 0;
    for (int x : arr) ans ^= x;
    return ans;
}
```

⏱ Complexity

- **Time:** O(N)
- **Space:** O(1)

Optimal Approach – Binary Search (O(log N))

⭐ Core Insight

Before the unique element:

Pairs are like:

```
Index:  0 1   2 3   4 5   6 ...
values: a a | b b | c c | ...
```

- **Every pair starts at even index.**

After the unique element:

Pattern shifts:

```
0 1  2 3  4 5  6  7
a a  b b  x c c  d d
         ↑
single element
```

After the single element,
pairs now start at **odd** index.

⭐ Binary Search Strategy

Let's use this pairing rule:

For any index `mid`:

- **If mid is even:**
  - Correct pair → `arr[mid] == arr[mid+1]`
- **If mid is odd:**
  - Correct pair → `arr[mid] == arr[mid-1]`

If pairing is correct → unique is on the **right side**
If pairing breaks → unique is on the **left side**

This gives O(log N) binary elimination.

```
int singleNonDuplicate(vector<int>& arr) {

    int n = arr.size();



    if (n == 1) return arr[0];

    if (arr[0] != arr[1]) return arr[0];

    if (arr[n-1] != arr[n-2]) return arr[n-1];



    int low = 1, high = n - 2;
```

```
    while (low <= high) {
        int mid = (low + high) / 2;

        // Direct check if mid is unique
        if (arr[mid] != arr[mid+1] && arr[mid] != arr[mid-1])
            return arr[mid];

        bool isPairCorrect =
            (mid % 2 == 0 && arr[mid] == arr[mid+1]) ||
            (mid % 2 == 1 && arr[mid] == arr[mid-1]);

        if (isPairCorrect) {
            low = mid + 1;      // unique lies right
        } else {
            high = mid - 1;     // unique lies left
        }
    }
    return -1; // won't happen
}
```

⏱ **Complexity**

Time → O(log N)

Binary search, discarding half each step.

Space → O(1)

No extra memory.

13. Peak element in Array

A **peak element** is one that is **greater than both neighbors**:

`arr[i-1] < arr[i] > arr[i+1]`

At boundaries:

- `i = 0` → only check right neighbor
- `i = n-1` → only check left neighbor

If multiple peaks exist → return **any one**.

Examples:

```
[1,2,3,4,5,6,7,8,5,1] → peak = 8 at index 7
[1,2,1,3,5,6,4]       → peaks at indices 1 and 5 → return any
```

🚀 2) Brute Force Approach (O(N))

Just check each element and see if it's greater than its neighbors.

```
int findPeakElement(vector<int>& nums) {
    int n = nums.size();

    for (int i = 0; i < n; i++) {
        bool left  = (i == 0) || (nums[i] >= nums[i - 1]);
        bool right = (i == n - 1) || (nums[i] >= nums[i + 1]);

        if (left && right)
            return i;
    }
    return -1; // shouldn't happen
}
```

✔ Complexity

- **Time:** O(N)
- **Space:** O(1)

OPTIMAL APPROACH – Binary Search (O(log N))

Think of the array like mountains—if you are **going uphill**, a peak **must exist somewhere ahead**.

Let's analyze:

Case A: `nums[mid] < nums[mid + 1]`

This means:

`mid → mid+1 → increasing slope`

So the peak lies **to the right**.

We move:

```
low = mid + 1
```

Case B: `nums[mid] > nums[mid + 1]`

This means:

`mid → mid+1 → decreasing slope`

So we are **in the right half of a mountain**,
and the peak lies at **mid OR left side**.

We move:

```
high = mid
```

❗ **What guarantees correctness?**

Every array with finite ends must have at least 1 peak.

Also:

- Left of peak: strictly increasing region
- Right of peak: strictly decreasing region

Binary search always moves toward a peak.

```
int findPeakElement(vector<int>& nums) {
    int low = 0, high = nums.size() - 1;
    while (low < high) {
        int mid = (low + high) / 2;
        // Descending? → peak is on left side (or mid)
        if (nums[mid] > nums[mid + 1]) {
            high = mid;
        }
        // Ascending? → peak is on right side
        else {
            low = mid + 1;
        }
    }
    return low; // low == high gives a peak index
}
```

Complexity

Operation	Complexity
Time	**O(log N)** (binary search halves array each time)
Space	**O(1)** (no extra memory)

14. Finding Sqrt of a number using Binary Search

Given a positive integer n, return:

- `sqrt(n)` **if n is a perfect square**,
- otherwise return **floor(sqrt(n))**.

Input: 36 → Output: 6

Input: 28 → Output: 5 (since sqrt(28) = 5.29…)

Brute Force Approach (O(N))

Try every number from **1 to n**, and track the largest number whose square is ≤ n.

```
int floorSqrt(int n) {
    int ans = 0;
    for (int i = 1; i <= n; i++) {
        if (1LL * i * i <= n)
            ans = i;
        else
            break;
    }
    return ans;
}
```

⏱ **Complexity**

- **Time:** O(N)
- **Space:** O(1)

Optimal Approach — Binary Search (O(log N))

The answer (integer sqrt) must lie in the **range [1…n]**.

Since this range is **sorted**, we can do binary search on it.

At each step:

- Compute `mid`.
- Compare `mid*mid` with n.
- If `mid*mid <= n` → mid is a *valid* square root → move right to find a bigger possible answer.
- If `mid*mid > n` → mid is too large → move left.

We keep track of the last valid `mid` as the answer.

```
int mySqrt(int x) {
    if (x < 2) return x;  // sqrt(0)=0, sqrt(1)=1
    int left = 1, right = x / 2;
    int ans = 0;
    while (left <= right) {
        long long mid = left + (right - left) / 2;
        if (mid * mid <= x) {
            ans = mid;        // mid is a possible answer
            left = mid + 1;   // try to find a larger one
        } else {
            right = mid - 1;  // mid is too large
        }
    }
    return ans;
}
```

15. Nth Root of a Number using Binary Search

Given two integers **N** and **M**, find an integer **X** such that:

$X^n = M$

If such an integer X does **not** exist → return **-1**.

Examples

```
N = 3, M = 27 → Output: 3

N = 4, M = 69 → Output: -1   (no integer 4th root)
```

Brute Force Approach — O(M)

Try every number from **1 to M** and check if $i^n == M$.

Stop if $i^n > M$.

```
int nthRoot(int n, int m) {
    for (int i = 1; i <= m; i++) {
        long long power = pow(i, n);
        if (power == m) return i;
        if (power > m) break;
    }
    return -1;}
```

Complexity

- **Time:** O(M)
- **Space:** O(1)

OPTIMAL APPROACH — Binary Search (O(log M))

The N-th root of M must lie in the range:

`1 to M`

Since this range is sorted, we can use **binary search**.

For a given mid:

Compute:

mid^n

We do NOT use `pow(mid, n)` because it:

- is slow
- causes overflow
- is inaccurate for big numbers

Instead, we multiply mid by itself n times **with overflow checking**.

⭐ **Helper function logic**

We create a function that compares:

mid^n with M

without overflowing.

```
#include <bits/stdc++.h>

using namespace std;

class Solution {

public:

    // Helper: compute mid^n with early break

    long long power(long long mid, int n, int m) {
```

```
    long long ans = 1;
    for (int i = 0; i < n; i++) {
        ans *= mid;
        if (ans > m) return ans;  // avoid overflow
    }
    return ans;
}

// Function to find Nth root of M
int nthRoot(int n, int m) {
    int low = 1, high = m;
    while (low <= high) {
        int mid = (low + high) / 2;

        long long val = power(mid, n, m);

        if (val == m) return mid;      // perfect nth root
        else if (val < m)
            low = mid + 1;            // try bigger
        else
            high = mid - 1;           // try smaller
    }
    return -1; // no integer root
} };
```

```
int main() {
    Solution sol;
    cout << sol.nthRoot(3, 27); // Output: 3
}
```

16. Koko Eating Bananas

Given n piles `piles[]` and an integer h (hours). Each hour Koko picks one non-empty pile and eats up to k bananas from it (if fewer than k remain, he finishes that pile and the hour ends).

Find the **minimum integer k** so Koko can finish all bananas within h hours.

If a speed k works (finishes in ≤ h hours), then every k' > k also works. So the answer lies in `[1, max(piles)]` and we can binary-search that answer space. For a candidate k compute total hours `sum( ceil(pile/k) )`.

Important edge case

Because each pile requires at least one hour, total hours ≥ n. If h < n, **no possible k** can finish (each pile requires distinct hour) — return `-1` (or handle per problem spec). I include that check in the implementations.

Key micro-optimizations

- Use integer formula `hours += (pile + k - 1) / k` to compute `ceil(pile/k)` using integers (no floats).
- Early stop while summing hours if `hours > h` — saves work for slow candidates.
- Use `long long` for hour sums to avoid overflow on large inputs.

```
#include <bits/stdc++.h>
```

```
using namespace std;

class Solution {
public:
    // Returns true if speed 'k' allows finishing within 'h' hours
    bool canFinish(const vector<int>& piles, int k, long long h) {
        long long hours = 0;
        for (int p : piles) {
            hours += (p + k - 1) / k;        // integer ceil
            if (hours > h) return false;    // early exit
        }
        return hours <= h;
    }

    // Returns minimum k, or -1 if impossible (h < n)
    int minEatingSpeed(vector<int>& piles, long long h) {
        int n = piles.size();
        if (h < n) return -1; // impossible: each pile needs at least one hour

        int lo = 1;
        int hi = *max_element(piles.begin(), piles.end());
        int ans = hi;

        while (lo <= hi) {
```

```
            int mid = lo + (hi - lo) / 2;
            if (canFinish(piles, mid, h)) {
                ans = mid;
                hi = mid - 1; // try smaller speed
            } else {
                lo = mid + 1; // need larger speed
            }
        }
        return ans;
    }
};
int main() {
    Solution s;
    vector<int> piles1 = {7, 15, 6, 3};
    cout << s.minEatingSpeed(piles1, 8) << '\n'; // 5

    vector<int> piles2 = {25, 12, 8, 14, 19};
    cout << s.minEatingSpeed(piles2, 5) << '\n'; // 25

    // Example impossible case:
    vector<int> piles3 = {3,6,7,11};
    cout << s.minEatingSpeed(piles3, 3) << '\n'; // -1 (since n=4 > h=3)
    return 0;
}
```

17. Minimum days to make M bouquets

You have:

- N roses
- An array `arr[]` where `arr[i]` = the day the *i-th* rose will **bloom**
- You need to make **m bouquets**
- Each bouquet needs **k adjacent bloomed roses**
- You can use a rose only if it is **already bloomed** (its bloom-day ≤ today).

Your goal:
👉 **Find the minimum day when you can form at least `m` bouquets.**
👉 If it is impossible, return `-1`.

What does “adjacent bloomed roses” mean?

If `k = 3`, you need **3 consecutive roses that have bloomed**.

For example:

```
arr = [7,7,7,7,13,11,12,7]
```
These are bloom days:

Index: 0 1 2 3 4 5 6 7
Day: 7 7 7 7 13 11 12 7

⭐ Example

Input:

```
N = 8

arr = {7, 7, 7, 7, 13, 11, 12, 7}

m = 2 (bouquets)

k = 3 (adjacent roses needed)
```

We must make **2 bouquets**, each having **3 adjacent roses**.

Let's check day = 12:

Index	Bloom day	Is bloomed by day 12?
0	7	✔
1	7	✔
2	7	✔
3	7	✔
4	13	✘
5	11	✔
6	12	✔
7	7	✔

Adjacent bloomed groups:

- [7 , 7 , 7 , 7] → from index 0–3 → **3 adjacent roses available** → 1 bouquet
- [11 , 12 , 7] → indexes 5–7 → **3 adjacent roses** → 1 bouquet

So, **2 bouquets possible on day 12** → ✔

But on day 11, rose at index 6 (12) isn't bloomed yet → ❌ cannot make two bouquets.

Thus answer = **12**.

❌ Brute Force

1. Check every possible day from `min(arr)` to `max(arr)`.
2. For each day, check if we can form `m` bouquets.
3. The first day that works → answer.

Time Complexity = O(N × (max-min)) → too large

✅ Optimal Approach (Binary Search on Days)

The answer (minimum day) lies between:

- **low = minimum bloom day**
- **high = maximum bloom day**

If Koko can make bouquets on day X,
➡️ She can definitely make them on any day > X.

This monotonic property ⇒ **Binary Search**.

Step 1 — Check impossible case

If `m × k > N`, immediately return `-1`.

K adjacent rose, and aise aise M chaiye

Step 2 — Binary search on days

```
low = min(arr)
high = max(arr)
```

While `low <= high`:

1. `mid = (low + high) / 2` → guess a day
2. Check `possible(arr, mid, m, k)`
3. If possible → try smaller day (high = mid - 1)
4. If not possible → need more days (low = mid + 1)

Final answer → low

Helper Function: **possible(day):**

If arr[i] <= day → rose bloomed

Else → reset count (not adjacent anymore)

Each time we find k consecutive bloomed roses, we increment bouquet count.

If bouquets >= m, return true.

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    // Check if it is possible to make m bouquets on or before 'day'

    bool possible(vector<int>& arr, int day, int m, int k) {

        int count = 0;

        int bouquets = 0;


        for (int bloom : arr) {

            if (bloom <= day) {

                count++;

                if (count == k) {
```

```
                bouquets++;
                count = 0;
            }
        } else {
            count = 0;
        }
    }
    return bouquets >= m;
}

// Main function
int minDays(vector<int>& arr, int m, int k) {
    long long needed = 1LL * m * k;
    if (needed > arr.size()) return -1;   // Not enough roses

    int low = *min_element(arr.begin(), arr.end());
    int high = *max_element(arr.begin(), arr.end());
    int ans = -1;

    while (low <= high) {
        int mid = (low + high) / 2;
        if (possible(arr, mid, m, k)) {
            ans = mid;
```

```
                high = mid - 1;     // try earlier day
            } else {
                low = mid + 1;      // need more days
            }
        }
        return ans;
    }
};
```

Time Complexity:

O(N × log(max(arr) - min(arr)))

Because:

- checking possible() takes O(N)
- binary search takes log(range of days)

Space Complexity:

O(1)

18. Find the Smallest Divisor Given a Threshold

You are given:

- An integer array arr
- A threshold value limit

You must choose **a positive divisor d** such that:

upon dividing all the elements of the given array by it, the sum of the division's result is less than or equal to the given threshold value.

Your objective:

👉 **Find the smallest such divisor d.**

If divisor = 1, the sum becomes extremely large.
If divisor increases, the sum decreases.

This monotonic behavior ⇒ **Binary Search on the divisor.**

✔ **Example 1 (Explained)**

Input:

arr = {1,2,3,4,5}, limit = 8

Check divisor = 1

ceil values =
1, 2, 3, 4, 5 → sum = **15**, too big ❌

Check divisor = 2

ceil(1/2)=1
ceil(2/2)=1
ceil(3/2)=2

ceil(4/2)=2
ceil(5/2)=3
Sum = **9**, still bigger ❌

Check divisor = 3

ceil(1/3)=1
ceil(2/3)=1
ceil(3/3)=1
ceil(4/3)=2
ceil(5/3)=2
Sum = **7**, OKAY ✔

But is there smaller divisor?
d = 2 → 9 > 8 ❌
So **d = 3** is the smallest.

❌ Brute Force Idea (very slow)

1. Try every divisor d from 1 to max(arr[]).
2. Compute the sum of ceil(arr[i]/d).
3. Return smallest d satisfying condition.

Time Complexity:

`O(max(arr) * N)  → too slow for large values`

✅ Optimal Solution: Binary Search on Answer

As divisor increases → sum decreases
This is a **monotonic function**, perfect for binary search.
low = 1
high = max(arr)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    // Helper to compute sum of ceilings
    int computeSum(vector<int>& arr, int d) {
```

```
        int sum = 0;
        for (int x : arr) {
            sum += (x + d - 1) / d;   // integer ceil
        }
        return sum;
    }

    int smallestDivisor(vector<int>& arr, int limit) {
        int n = arr.size();

        // If elements > limit, sum of ceil(arr[i]/d) can never be <= limit for d>=1
        if (n > limit) return -1;

        int low = 1;
        int high = *max_element(arr.begin(), arr.end());
        int ans = high;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (computeSum(arr, mid) <= limit) {
                ans = mid;       // possible divisor
                high = mid - 1;  // try smaller
            }
            else {
                low = mid + 1;   // need larger divisor
            }
        }

        return ans;
    }
};
```

19. Capacity to Ship Packages within D Days

You own a shipping company.

- You have a list of package weights (must be shipped **in order**).
- You have **D days** to ship them.
- Each day you load packages **in order**, but the ship **cannot exceed capacity**.
- If capacity is too small, it will take many days.
- If capacity is huge, everything ships in 1 day.

👉 Your task: **Find the minimum ship capacity so that all packages are shipped in ≤ D days.**

⭐ **Example 1**

weights = [5,4,5,2,3,4,5,6]
d = 5

We want the **minimum** capacity that allows shipping in **5 days**.

Let's test capacity = **9** (the answer):

Day-wise loading:

- **Day 1**: 5+4 = 9 (stop)
- **Day 2**: 5+2+3 = 10 > 9 → only 5,2
- **Day 3**: 3+4 = 7
- **Day 4**: 5
- **Day 5**: 6

Total = 5 days ⇒ **works**

Now check capacity = 8:

- Day 1: 5+4 = 9 > 8 ⇒ only 5
- Day 2: 4+5 = 9 > 8
  … and you will use more than 5 days.

So **8 doesn't work**, **9 is the smallest**.

🧠 Brute Force Approach:

Idea:

Try every possible capacity from
`max(weights)` → `sum(weights)`

For each capacity:

- Simulate shipping
- Count number of days taken
- First capacity that fits in D days is the answer

Correct but very slow.

❌ Brute Force: Time Complexity

```
O(N * (sum(weights) - max(weights)))
```

Too slow (sum(weights) can be up to 10^9).

✅ Optimal Approach (Binary Search)

💡 Key Insight:

The answer lies between:

- **min capacity = max(weights)**
  (because we cannot split a package)

- **max capacity = sum(weights)**
  (ship everything in 1 day)

As capacity increases → days required decreases
This is **monotonic**, so we use **Binary Search on capacity**.

low = max(weights)

high = sum(weights)

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    // How many days needed if ship capacity = cap?

    int daysNeeded(vector<int>& weights, int cap) {

        int days = 1;

        int load = 0;


        for (int w : weights) {

            if (load + w > cap) {

                days++;

                load = w;

            } else {

                load += w;

            }
```

```
        }
        return days;
    }

    int shipWithinDays(vector<int>& weights, int d) {
        int low = *max_element(weights.begin(), weights.end());
        int high = accumulate(weights.begin(), weights.end(), 0);

        while (low < high) {
            int mid = low + (high - low) / 2;

            if (daysNeeded(weights, mid) <= d)
                high = mid;        // try smaller
            else
                low = mid + 1;     // need larger capacity
        }

        return low;
    }
};
```

20. Kth Missing Positive Number

You are given a strictly increasing array `vec` of positive integers and a positive integer `k`.
Find the **k-th positive integer that is missing** from `vec`.
(Example: if `vec = [4,7,9,10]`, the missing positives are `1,2,3,5,6,8,11,...`)

For index `i` (0-based), the number of missing positive integers **before** `vec[i]` is:

```
missing_before_i = vec[i] - (i + 1)
```

Because if there were no missing numbers the value at index `i` would be `i+1`.
Use this to locate where the k-th missing number lies.

Approach A — Brute force (simple)

Walk the array and for each element `a = vec[i]`:

- while `k >= a - current` (where `current` is the next candidate missing), skip forward; simpler implementation: increment k for each `vec[i] <= k`.
  A clearer and compact brute implementation is:

```
// Brute (O(n))
int missingK_brute(const vector<int>& vec, int k) {
    for (int x : vec) {
        if (x <= k) k++;
        else break;
    }
    return k;
}
```

Why it works: If `x <= k` then x "uses up" a missing number ≤ current k, so the k-th missing shifts up by 1.

Complexity: O(n), O(1) space.

Approach B — Optimal (binary search) — O(log n)

Use the formula `missing_before_i = vec[i] - (i+1)`.

If `missing_before_last < k` then the answer is beyond the last element:

```
answer = vec.back() + (k - missing_before_last)
```

Otherwise binary search for the largest index `high` such that `missing_before_high < k`. Then the k-th missing is:

```
answer = (high + 1) + k

// equivalently: k + high + 1

// or vec[high] + (k - missing_before_high)
```

Reason

At `high` we have fewer than k missing numbers. The k-th missing lies between `vec[high]` and `vec[high+1]` (or after the array if `high` is last index). The formula `k + high + 1` yields the exact missing integer.

```
#include <bits/stdc++.h>
using namespace std;

int findKthMissing(const vector<int>& vec, int k) {
    int n = vec.size();
    int low = 0, high = n - 1;

    // if total missing until last element is < k, it's after the array
    int missingLast = vec.back() - n; // vec[n-1] - (n)
    if (missingLast < k) {
        return vec.back() + (k - missingLast);
    }

    // binary search for largest index with missing_before < k
    while (low <= high) {
```

```
        int mid = low + (high - low) / 2;
        int missingBeforeMid = vec[mid] - (mid + 1);
        if (missingBeforeMid < k) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    // high is the largest index with missing_before_high < k
    // answer = k + high + 1
    return k + high + 1;
}

int main() {
    vector<int> vec = {4,7,9,10};
    cout << findKthMissing(vec, 1) << "\n"; // 1
    cout << findKthMissing(vec, 4) << "\n"; // 5
    return 0;
}
```

Complexity

- **Binary search approach:** O(log n) time, O(1) space.
- **Brute approach (scan):** O(n) time, O(1) space.

21. Aggressive Cows

Array me stalls ke positions diye hote hain aur k cows ko aise place karna hota hai ki kisi bhi do cows ke beech ka **minimum distance maximum** ho.

Core Idea

Direct maximum distance nikalna mushkil hai, lekin hum ye check kar sakte hain ki
kya minimum distance d rakh kar saari cows place ho sakti hain ya nahi.

Agar:

- distance d possible hai → usse chhota bhi possible hoga
- distance d possible nahi hai → usse bada bhi possible nahi hoga

Isliye answer space **monotonic** hai → Binary Search lagegi.

Search Space

- minimum distance = 1
- maximum distance = `max(stalls) - min(stalls)`

```
low = 1
high = stalls[last] - stalls[first]
```

---

canPlace(d) logic

- First cow ko first stall me rakh do

Har next stall par check karo:

```
stalls[i] - lastPlaced >= d
```

-
- Agar haan → cow place karo
- Jaise hi k cows place ho jaaye → true
- End tak nahi ho paayi → false

Greedy approach hai.

Binary Search Flow

```
while(low <= high):
    mid = (low + high) / 2

    if canPlace(mid):
        answer = mid
        low = mid + 1     // distance aur badhane ki koshish
    else:
        high = mid - 1    // distance kam karo
```

Loop ke baad `answer` hi final maximum minimum distance hoga.

C++ Code

```
bool canPlace(vector<int>& stalls, int cows, int d) {
    int count = 1;
    int lastPos = stalls[0];

    for (int i = 1; i < stalls.size(); i++) {
        if (stalls[i] - lastPos >= d) {
            count++;
            lastPos = stalls[i];
        }
        if (count >= cows) return true;
    }
    return false;
}

int aggressiveCows(vector<int>& stalls, int cows) {
    sort(stalls.begin(), stalls.end());

    int low = 1;
    int high = stalls.back() - stalls.front();
    int ans = 0;
```

```
    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (canPlace(stalls, cows, mid)) {
            ans = mid;
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return ans;
}
```

Complexity

- Sorting: `O(N log N)`
- Binary Search × check: `O(N log(maxDist))`
- Space: `O(1)`

22. Allocate Minimum Number of Pages

You are given:

- An integer array `arr` where `arr[i]` denotes the number of pages in the `i-th` book
- An integer `m` representing the number of students

You must allocate **all books** to the students such that:

1. Each student gets **at least one book**
2. Each book is allocated to **exactly one student**
3. Books allocated to a student must be **contiguous**
4. The **maximum number of pages assigned to any student is minimized**

If it is not possible to allocate books under these conditions, return `-1`.

Example 1

```
Input:

arr = [12, 34, 67, 90]

m = 2


Output:

113


Explanation:

Possible allocation:

Student 1 → [12, 34, 67] → 113 pages

Student 2 → [90] → 90 pages


Maximum pages = 113 (minimum possible)
```

Example 2

```
Input:

arr = [25, 46, 28, 49, 24]

m = 4


Output:

71


Explanation:

Allocation:

[25, 46] | [28] | [49] | [24]


Maximum pages = 71
```

Key Observation

We cannot directly find the optimal allocation, but we can **check feasibility**.

Question to ask:

> If each student is allowed to read **at most X pages**, can we allocate all books using **≤ m students**?

- If more than `m` students are required → X is too small
- If `m` or fewer students are sufficient → X is valid

This feasibility condition is monotonic → **Binary Search applies**.

Search Space

- Minimum pages = `max(arr[])`
  (a student must read at least one full book)

- Maximum pages = `sum(arr[])`
  (one student reads all books)

```
low = max(arr)
high = sum(arr)
```

---

countStudents(pages)

Greedy allocation:

- Start with one student

- Add books to the current student until page limit exceeds

- Then assign next book to a new student

```
students = 1
currentPages = 0

for each book:
    if currentPages + book <= pages:
        currentPages += book
    else:
        students++
        currentPages = book
```

Return the number of students needed.

Binary Search Logic

```
if m > n: return -1

while(low <= high):
    mid = (low + high) / 2
    students = countStudents(mid)

    if students > m:
        low = mid + 1      // pages too small
    else:
        high = mid - 1     // try smaller maximum
```

After the loop, `low` gives the **minimum possible maximum pages**.

C++ Code

```
int countStudents(vector<int> &arr, int pages) {
    int students = 1;
    long long currentPages = 0;

    for (int i = 0; i < arr.size(); i++) {
        if (currentPages + arr[i] <= pages) {
            currentPages += arr[i];
        } else {
            students++;
            currentPages = arr[i];
        }
    }
    return students;
}

int findPages(vector<int>& arr, int n, int m) {
    if (m > n) return -1;

    int low = *max_element(arr.begin(), arr.end());
    int high = accumulate(arr.begin(), arr.end(), 0);
```

```
    while (low <= high) {
        int mid = low + (high - low) / 2;
        int students = countStudents(arr, mid);

        if (students > m) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return low;
}
```

Example Insight

```
arr = [12, 34, 67, 90], m = 2

Minimum max pages = 113
Allocation: [12, 34, 67] | [90]
```

Complexity

- Feasibility check: `O(N)`
- Binary Search steps: `O(log(sum - max))`

Total Time: `O(N log(sum(arr) - max(arr)))`
Space: `O(1)`

23. Split Array – Largest Sum

Problem Statement

You are given an integer array `A` of size `N` and an integer `K`.

Split the array into **exactly K non-empty subarrays** such that:

- Each subarray is **contiguous**
- Every element belongs to exactly one subarray
- The **largest sum among all subarrays is minimized**

Return the minimized largest subarray sum.

Example 1

```
Input:
A = [1, 2, 3, 4, 5]
K = 3

Output:
6

Explanation:
Split as:
[1, 2, 3] | [4] | [5]

Subarray sums = 6, 4, 5
Largest sum = 6
```

Example 2

```
Input:
A = [3, 5, 1]
```

```
K = 3

Output:
5

Explanation:
Split as:
[3] | [5] | [1]

Largest sum = 5
```

Approach (Binary Search on Answer)

Key Idea

Instead of deciding the split directly, decide the **maximum allowed subarray sum** and check if the array can be split into **at most K subarrays**.

- If more than K subarrays are required → allowed sum is too small
- If K or fewer subarrays are enough → allowed sum is valid

The answer space is monotonic, so Binary Search applies.

Search Space

- `low = max(A)`
- `high = sum(A)`

Feasibility Function

Greedy splitting:

- Keep adding elements to the current subarray
- If adding an element exceeds the allowed sum, start a new subarray

```
partitions = 1
currentSum = 0
```

Binary Search Logic

```
while(low <= high):
    mid = (low + high) / 2
    partitions = countPartitions(mid)

    if partitions > K:
        low = mid + 1
    else:
        high = mid - 1
```

Final answer is `low`.

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

int countPartitions(vector<int>& a, int maxSum) {
    int partitions = 1;
    long long currentSum = 0;

    for (int i = 0; i < a.size(); i++) {
        if (currentSum + a[i] <= maxSum) {
            currentSum += a[i];
        } else {
            partitions++;
            currentSum = a[i];
        }
```

```
    }
    return partitions;
}

int splitArrayLargestSum(vector<int>& a, int k) {
    int low = *max_element(a.begin(), a.end());
    int high = accumulate(a.begin(), a.end(), 0);

    while (low <= high) {
        int mid = low + (high - low) / 2;
        int partitions = countPartitions(a, mid);

        if (partitions > k) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return low;
}

int main() {
    vector<int> a = {1, 2, 3, 4, 5};
    int k = 3;
    cout << splitArrayLargestSum(a, k) << endl;
    return 0;
}
```

Complexity

- Time: `O(N log(sum - max))`
- Space: `O(1)`

24. Painter’s Partition Problem

Problem Statement

You are given an array `boards` of length `N`, where `boards[i]` represents the length of the `i-th` board.
There are `K` painters available. Each painter paints at a speed of **1 unit length per unit time**.

Constraints:

- Each painter can paint **only contiguous boards**
- Each board is painted by **exactly one painter**
- All boards must be painted

Return the **minimum time** required to paint all boards.

Example 1

```
Input:
boards = [5, 5, 5, 5]
K = 2

Output:
10

Explanation:
Split as:
[5, 5] | [5, 5]

Each painter paints 10 units → time = 10
```

Example 2

```
Input:
boards = [10, 20, 30, 40]
```

```
K = 2

Output:
60

Explanation:
Split as:
[10, 20, 30] | [40]

Painter times = 60, 40
Maximum = 60 (minimum possible)
```

Approach

Key Idea

Instead of deciding which painter paints which boards, decide the **maximum time** a painter is allowed to work and check if the job can be completed using at most K painters.

Question to check:

> If each painter can work for **at most T time**, can all boards be painted using **≤ K painters**?

- If more than K painters are needed → T is too small
- If K or fewer painters are enough → T is valid

This condition is monotonic, so Binary Search applies.

Search Space

- Minimum time = `max(boards)`
  (a painter must paint at least one full board)

- Maximum time = `sum(boards)`
  (one painter paints all boards)

```
low = max(boards)
high = sum(boards)
```

Feasibility Function: countPainters(time)

Greedy allocation:

- Assign boards in order to the current painter
- If adding a board exceeds `time`, assign it to a new painter

```
painters = 1
currentWork = 0
```

Binary Search Logic

```
while(low <= high):
    mid = (low + high) / 2
    painters = countPainters(mid)

    if painters > K:
        low = mid + 1      // time too small
    else:
        high = mid - 1     // try smaller time
```

Final answer is `low`.

C++ Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
int countPainters(vector<int>& boards, int time) {
    int painters = 1;
    long long currentWork = 0;

    for (int i = 0; i < boards.size(); i++) {
        if (currentWork + boards[i] <= time) {
            currentWork += boards[i];
        } else {
            painters++;
            currentWork = boards[i];
        }
    }
    return painters;
}

int painterPartition(vector<int>& boards, int k) {
    int low = *max_element(boards.begin(), boards.end());
    int high = accumulate(boards.begin(), boards.end(), 0);

    while (low <= high) {
        int mid = low + (high - low) / 2;
        int painters = countPainters(boards, mid);

        if (painters > k) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return low;
}

int main() {
    vector<int> boards = {10, 20, 30, 40};
    int k = 2;
    cout << painterPartition(boards, k) << endl;
    return 0;
```

```
}
```

Complexity

- Feasibility check: `O(N)`
- Binary Search: `O(log(sum - max))`

Total Time: `O(N log(sum - max))`
Space: `O(1)`

25. Minimise Maximum Distance between Gas Stations

Problem Statement

You are given a **sorted** array `arr` of size n, where `arr[i]` represents the position of the `i-th` gas station on the X-axis.
You are also given an integer k, representing the number of **new gas stations** to be added.

You can place new gas stations at **any non-negative position**, including non-integer positions.

After adding k gas stations, let `dist` be the **maximum distance between any two adjacent gas stations**.
Your task is to **minimize this maximum distance** and return its value.

Example 1

```
Input:
arr = [1, 2, 3, 4, 5]
k = 4
```

```
Output:
0.5

Explanation:
One possible placement:
[1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5]

Maximum adjacent distance = 0.5
```

Example 2

```
Input:
arr = [1,2,3,4,5,6,7,8,9,10]
k = 1

Output:
1

Explanation:
Best placement does not reduce the maximum distance below 1.
```

Approach (Binary Search on Answer)

Key Idea

Instead of deciding exact positions for new gas stations, we decide the **maximum allowed distance `D`** between adjacent stations and check feasibility.

Question to check:

> If the maximum allowed distance is `D`, how many gas stations are required to ensure no segment exceeds `D`?

- If **required stations > k** → `D` is too small

- If **required stations ≤ k** → `D` is valid

This feasibility condition is monotonic → Binary Search applies.

Search Space

- `low = 0`
- `high = maximum distance between consecutive existing stations`

```
high = max(arr[i+1] - arr[i])
```

Feasibility Function

For each gap `gap = arr[i] - arr[i-1]`:

Number of stations needed so that each segment ≤ `D`:

```
stations = floor(gap / D)
```

If `gap` is exactly divisible by `D`, subtract 1 to avoid placing an extra station.

Sum this over all gaps.

Binary Search Logic

Binary search is done on real numbers, so we stop when precision is small enough.

```
while(high - low > 1e-6):
    mid = (low + high) / 2

    if requiredStations(mid) > k:
        low = mid
    else:
```

```
        high = mid
```

Final answer can be high.

C++ Code (Optimal Solution)

```
#include <bits/stdc++.h>
using namespace std;

int numberOfGasStationsRequired(long double dist, vector<int>& arr) {
    int cnt = 0;
    for (int i = 1; i < arr.size(); i++) {
        long double gap = arr[i] - arr[i - 1];
        int stations = gap / dist;
        if (gap == dist * stations) stations--;
        cnt += stations;
    }
    return cnt;
}

long double minimiseMaxDistance(vector<int>& arr, int k) {
    long double low = 0, high = 0;

    for (int i = 0; i < arr.size() - 1; i++) {
        high = max(high, (long double)(arr[i + 1] - arr[i]));
    }

    long double precision = 1e-6;

    while (high - low > precision) {
        long double mid = (low + high) / 2.0;
        if (numberOfGasStationsRequired(mid, arr) > k)
            low = mid;
        else
            high = mid;
    }
    return high;
```

```
}

int main() {
    vector<int> arr = {1, 2, 3, 4, 5};
    int k = 4;
    cout << minimiseMaxDistance(arr, k) << endl;
    return 0;
}
```

Complexity

- Feasibility check: `O(n)`
- Binary Search iterations: `O(log(range / precision))`

Total Time: `O(n log(range))`
Space: `O(1)`

26. Median of Two Sorted Arrays of Different Sizes

Problem Statement

You are given two **sorted** arrays `arr1` and `arr2` of sizes n1 and n2.

Return the **median** of the combined sorted array formed after merging both arrays.

- If the total number of elements is **odd**, the median is the middle element.

- If the total number of elements is **even**, the median is the **average of the two middle elements**.

Example 1

```
Input:
arr1 = [2, 4, 6]
arr2 = [1, 3, 5]

Output:
3.5

Explanation:
Merged array = [1, 2, 3, 4, 5, 6]
Median = (3 + 4) / 2 = 3.5
```

Example 2

```
Input:
arr1 = [2, 4, 6]
arr2 = [1, 3]

Output:
3

Explanation:
Merged array = [1, 2, 3, 4, 6]
Median = 3
```

Optimal Approach (Binary Search on Partition)

Key Idea

Instead of merging the arrays, we **partition** them such that:

- Left half contains exactly half (or half + 1) elements
- All elements in left half ≤ all elements in right half

The median lies at the boundary of these partitions.

Binary search is applied on the **smaller array** to minimize time complexity.

Partition Conditions

Let:

- `cut1` = number of elements taken from `arr1`
- `cut2` = number of elements taken from `arr2`

```
cut1 + cut2 = (n1 + n2 + 1) / 2
```

Let:

- `l1` = last element on left of arr1
- `r1` = first element on right of arr1
- `l2` = last element on left of arr2
- `r2` = first element on right of arr2

Correct partition condition:

```
l1 <= r2 AND l2 <= r1
```

Median Calculation

If `(n1 + n2)` is even:

```
median = (max(l1, l2) + min(r1, r2)) / 2
```

-

If odd:

```
median = max(l1, l2)
```

-

C++ Code (Optimal)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    double findMedianSortedArrays(vector<int>& a, vector<int>& b) {
        // Ensure binary search on smaller array
        if (a.size() > b.size())
            return findMedianSortedArrays(b, a);

        int n1 = a.size(), n2 = b.size();
        int low = 0, high = n1;

        while (low <= high) {
            int cut1 = (low + high) / 2;
            int cut2 = (n1 + n2 + 1) / 2 - cut1;

            int l1 = (cut1 == 0) ? INT_MIN : a[cut1 - 1];
            int l2 = (cut2 == 0) ? INT_MIN : b[cut2 - 1];
            int r1 = (cut1 == n1) ? INT_MAX : a[cut1];
            int r2 = (cut2 == n2) ? INT_MAX : b[cut2];

            if (l1 <= r2 && l2 <= r1) {
                if ((n1 + n2) % 2 == 0)
                    return (max(l1, l2) + min(r1, r2)) / 2.0;
                else
                    return max(l1, l2);
            }
```

```
            else if (l1 > r2) {
                high = cut1 - 1;
            }
            else {
                low = cut1 + 1;
            }
        }
        return 0.0;
    }
};

int main() {
    vector<int> arr1 = {2, 4, 6};
    vector<int> arr2 = {1, 3};
    Solution sol;
    cout << sol.findMedianSortedArrays(arr1, arr2) << endl;
    return 0;
}
```

Complexity

- **Time:** `O(log(min(n1, n2)))`
- **Space:** `O(1)`

27. K-th Element of Two Sorted Arrays

Problem Statement

You are given two **sorted** arrays a and b of sizes `m` and `n`.
Find the **k-th element (1-based index)** in the final sorted array formed by merging both arrays.

Example 1

```
Input:
a = [2, 3, 6, 7, 9]
b = [1, 4, 8, 10]
k = 5

Output:
6

Explanation:
Merged array = [1, 2, 3, 4, 6, 7, 8, 9, 10]
5th element = 6
```

Example 2

```
Input:
a = [100, 112, 256, 349, 770]
b = [72, 86, 113, 119, 265, 445, 892]
k = 7

Output:
256
```

Approach (Binary Search on Partition)

Key Idea

This problem is an extension of **Median of Two Sorted Arrays**.

We partition both arrays such that:

- Total elements on the **left side = k**
- All elements on the left side ≤ all elements on the right side

Binary search is applied on the **smaller array** to optimize time.

Setup

Let:

- a be the smaller array (swap if needed)
- `left = k`

Binary search range:

```
low = max(0, k - n)
high = min(k, m)
```

Partition Logic

Choose:

```
mid1 = elements taken from a
mid2 = elements taken from b = k - mid1
```

Define boundary values:

```
l1 = (mid1 == 0) ? -∞ : a[mid1 - 1]
l2 = (mid2 == 0) ? -∞ : b[mid2 - 1]
r1 = (mid1 == m) ? +∞ : a[mid1]
r2 = (mid2 == n) ? +∞ : b[mid2]
```

Correct partition condition:

```
l1 <= r2 AND l2 <= r1
```

When satisfied:

```
Answer = max(l1, l2)
```

Binary Search Logic

```
while(low <= high):
    mid1 = (low + high) / 2
    mid2 = k - mid1

    if l1 <= r2 and l2 <= r1:
        return max(l1, l2)
    else if l1 > r2:
        high = mid1 - 1
    else:
        low = mid1 + 1
```

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int kthElement(vector<int>& a, vector<int>& b, int k) {
        int m = a.size(), n = b.size();

        // Always binary search on smaller array
        if (m > n) return kthElement(b, a, k);

        int low = max(0, k - n);
        int high = min(k, m);

        while (low <= high) {
            int mid1 = (low + high) / 2;
            int mid2 = k - mid1;

            int l1 = (mid1 == 0) ? INT_MIN : a[mid1 - 1];
            int l2 = (mid2 == 0) ? INT_MIN : b[mid2 - 1];
```

```
            int r1 = (mid1 == m) ? INT_MAX : a[mid1];
            int r2 = (mid2 == n) ? INT_MAX : b[mid2];

            if (l1 <= r2 && l2 <= r1) {
                return max(l1, l2);
            }
            else if (l1 > r2) {
                high = mid1 - 1;
            }
            else {
                low = mid1 + 1;
            }
        }
        return -1;
    }
};

int main() {
    vector<int> a = {2, 3, 6, 7, 9};
    vector<int> b = {1, 4, 8, 10};
    int k = 5;

    Solution sol;
    cout << sol.kthElement(a, b, k) << endl;
    return 0;
}
```

Complexity

- **Time:** $O(\log(\min(m, n)))$
- **Space:** $O(1)$

28. Find the Row with Maximum Number of 1’s

Problem Statement

You are given a non-empty binary matrix `mat` with n rows and `m` columns.
Each row is **sorted in ascending order** (all 0’s come before 1’s).

Your task is to return the **index (0-based)** of the row that contains the **maximum number of 1’s**.

Rules:

- If multiple rows have the same maximum number of 1’s, return the **smallest index**
- If the matrix contains **no 1 at all**, return `-1`

Example 1

```
Input:
mat =
1 1 1
0 0 1
0 0 0

Output:
0

Explanation:
Row 0 has 3 ones, which is the maximum.
```

Example 2

```
Input:
mat =
0 0
```

```
0 0

Output:
-1

Explanation:
No row contains a 1.
```

Brute Force Approach

Idea

Check every row and count the number of 1s manually using nested loops.

Steps

Initialize:

```
maxCount = 0
rowIndex = -1
```

1.
2. For each row:
    - Traverse all columns
    - Count number of 1s
3. If current count > maxCount:
    - Update `maxCount`
    - Update `rowIndex`
4. Return `rowIndex`

C++ Code (Brute Force)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int rowWithMax1s(vector<vector<int>>& mat, int n, int m) {
        int maxCount = 0;
        int rowIndex = -1;

        for (int i = 0; i < n; i++) {
            int countOnes = 0;
            for (int j = 0; j < m; j++) {
                countOnes += mat[i][j];
            }
            if (countOnes > maxCount) {
                maxCount = countOnes;
                rowIndex = i;
            }
        }
        return rowIndex;
    }
};
```

Complexity (Brute)

- **Time:** $O(n \times m)$
- **Space:** $O(1)$

Optimal Approach (Binary Search)

Each row is **sorted**, so:

- All 0’s appear before all 1’s

- If we find the **first occurrence of 1** in a row,
  then number of 1's = `m - index_of_first_1`

Instead of counting all elements, we use **Binary Search** on each row.

Steps

Initialize:

```
maxCount = 0
rowIndex = -1
```

1.
2. For each row:
    - Use binary search to find the first index where value is 1

Count number of 1's using:

```
ones = m - firstIndex
```

    -

If `ones > maxCount`, update:

```
maxCount = ones
rowIndex = currentRow
```

3.
4. Return `rowIndex`

Binary Search Logic (Lower Bound for 1)

Find the smallest index `i` such that `row[i] == 1`.

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int firstOneIndex(vector<int>& row, int m) {
        int low = 0, high = m - 1;
        int ans = m;

        while (low <= high) {
            int mid = (low + high) / 2;
            if (row[mid] == 1) {
                ans = mid;
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        }
        return ans;
    }

    int rowWithMax1s(vector<vector<int>>& mat, int n, int m) {
        int maxCount = 0;
        int rowIndex = -1;

        for (int i = 0; i < n; i++) {
            int firstOne = firstOneIndex(mat[i], m);
            int ones = m - firstOne;

            if (ones > maxCount) {
                maxCount = ones;
                rowIndex = i;
            }
        }
        return rowIndex;
```

```
    }
};

int main() {
    vector<vector<int>> mat = {
        {1, 1, 1},
        {0, 0, 1},
        {0, 0, 0}
    };

    Solution obj;
    cout << obj.rowWithMax1s(mat, 3, 3) << endl;
    return 0;
}
```

Complexity

- Binary search per row: `O(log m)`
- Total rows: n

Time Complexity: `O(n log m)`
Space Complexity: `O(1)`

29. Search in a Sorted 2D Matrix

Problem Statement

You are given a 2D matrix `mat` of size `N × M` such that:

- Each row is sorted in **non-decreasing order**

- The **first element of each row is greater than the last element of the previous row**

You are also given an integer `target`.
Return `true` if `target` exists in the matrix, otherwise return `false`.

Example 1

```
Input:
mat = [
  [1, 2, 3, 4],
  [5, 6, 7, 8],
  [9, 10, 11, 12]
]
target = 8

Output:
true
```

Example 2

```
Input:
mat = [
  [1, 2, 4],
  [6, 7, 8],
  [9, 10, 34]
]
target = 78

Output:
false
```

Brute Force Approach

Idea

Check every element of the matrix one by one.

Steps

1. Traverse each row
2. Traverse each column inside that row
3. If any element equals `target`, return `true`
4. If traversal ends, return `false`

C++ Code (Brute Force)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool searchMatrix(vector<vector<int>>& matrix, int target) {
        int n = matrix.size();
        int m = matrix[0].size();

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if (matrix[i][j] == target)
                    return true;
            }
        }
        return false;
    }
};
```

Complexity (Brute)

- **Time:** `O(N × M)`
- **Space:** `O(1)`

Better Approach (Binary Search Row-wise)

Idea

Each row is sorted.
First identify the row where the target **can possibly exist**, then apply binary search on that row.

A row `i` can contain `target` if:

```
mat[i][0] <= target <= mat[i][m-1]
```

Steps

1. Traverse rows one by one
2. For a row satisfying the above condition:
    - Apply binary search on that row
3. If found, return `true`
4. Otherwise, return `false`

C++ Code (Better)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
```

```
    bool binarySearch(vector<int>& row, int target) {
        int low = 0, high = row.size() - 1;

        while (low <= high) {
            int mid = (low + high) / 2;
            if (row[mid] == target)
                return true;
            else if (row[mid] < target)
                low = mid + 1;
            else
                high = mid - 1;
        }
        return false;
    }

    bool searchMatrix(vector<vector<int>>& matrix, int target) {
        int n = matrix.size();
        int m = matrix[0].size();

        for (int i = 0; i < n; i++) {
            if (matrix[i][0] <= target && target <= matrix[i][m - 1])
{
                return binarySearch(matrix[i], target);
            }
        }
        return false;
    }
};
```

Complexity (Better)

- **Time:** $O(N \times \log M)$
- **Space:** $O(1)$

Optimal Approach (Binary Search on Virtual 1D Array)

Idea

Because:

- Rows are sorted
- First element of a row > last element of previous row

The entire matrix behaves like a **single sorted 1D array**.

Instead of flattening, we **map indices**:

```
row = mid / M
col = mid % M
```

Steps

1. Consider search space from `0` to `N×M - 1`
2. Apply binary search on this imaginary array
3. Convert mid index to `(row, col)`
4. Compare `mat[row][col]` with `target`

C++ Code (Optimal)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool searchMatrix(vector<vector<int>>& matrix, int target) {
        int n = matrix.size();
        int m = matrix[0].size();
```

```
        int low = 0, high = n * m - 1;

        while (low <= high) {
            int mid = (low + high) / 2;
            int row = mid / m;
            int col = mid % m;

            if (matrix[row][col] == target)
                return true;
            else if (matrix[row][col] < target)
                low = mid + 1;
            else
                high = mid - 1;
        }
        return false;
    }
};
```

Complexity (Optimal)

- **Time:** $O(\log(N \times M))$
- **Space:** $O(1)$

When to Use Which

- Small matrix → Brute force
- Row-wise sorted only → Row-wise binary search
- Fully sorted matrix (this problem) → Virtual 1D binary search

30. Search in a Row and Column-wise Sorted Matrix

Problem Statement

You are given a 2D matrix `mat` of size `N × M` such that:

- Each **row** is sorted in non-decreasing order
- Each **column** is sorted in non-decreasing order
- The first element of a row is **not necessarily greater** than the last element of the previous row

You are given an integer `target`.
Your task is to determine whether `target` exists in the matrix or not.

Example 1

```
Matrix:
1   4   7   11
2   5   8   12
3   6   9   16
10 13  14  17

Target: 9
Output: Found at (2, 2)
```

Example 2

```
Matrix:
5   10  15
6   12  18
8   16  20

Target: 7
```

```
Output: Not Found
```

Brute Force Approach

Idea

Traverse the entire matrix and check each element.

Steps

1. Traverse row by row
2. Traverse column by column inside each row
3. If any element equals `target`, return `true`
4. If traversal ends, return `false`

C++ Code (Brute Force)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool searchMatrix(vector<vector<int>>& mat, int target) {
        int n = mat.size();
        int m = mat[0].size();

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if (mat[i][j] == target)
                    return true;
            }
```

```
        }
        return false;
    }
};
```

Complexity (Brute)

- **Time:** `O(N × M)`
- **Space:** `O(1)`

Better Approach (Binary Search on Each Row)

Idea

Each row is sorted, so instead of checking all columns:

- Apply **binary search** on each row

Steps

1. For each row:
    - Apply binary search to find `target`
2. If found in any row, return `true`
3. Otherwise, return `false`

C++ Code (Better)

```
#include <bits/stdc++.h>
```

```
using namespace std;

class Solution {
public:
    bool binarySearch(vector<int>& row, int target) {
        int low = 0, high = row.size() - 1;

        while (low <= high) {
            int mid = (low + high) / 2;
            if (row[mid] == target)
                return true;
            else if (row[mid] < target)
                low = mid + 1;
            else
                high = mid - 1;
        }
        return false;
    }

    bool searchMatrix(vector<vector<int>>& mat, int target) {
        int n = mat.size();

        for (int i = 0; i < n; i++) {
            if (binarySearch(mat[i], target))
                return true;
        }
        return false;
    }
};
```

Complexity (Better)

- **Time:** `O(N log M)`
- **Space:** `O(1)`

Optimal Approach (Staircase Search)

Idea

Start from the **top-right corner** `(0, M-1)`.

Why this works:

- Moving **left** decreases the value
- Moving **down** increases the value

So at every step, one full row or column gets eliminated.

Steps

1. Start at `(row = 0, col = M-1)`
2. While `row < N` and `col >= 0`:
    - If `mat[row][col] == target` → return `true`
    - If `mat[row][col] > target` → move left (`col--`)
    - If `mat[row][col] < target` → move down (`row++`)
3. If search ends, return `false`

C++ Code (Optimal)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool searchMatrix(vector<vector<int>>& mat, int target) {
        int n = mat.size();
```

```
        int m = mat[0].size();

        int row = 0, col = m - 1;

        while (row < n && col >= 0) {
            if (mat[row][col] == target)
                return true;
            else if (mat[row][col] > target)
                col--;
            else
                row++;
        }
        return false;
    }
};
```

Complexity (Optimal)

- **Time:** `O(N + M)`
- **Space:** `O(1)`

When to Use Which

- Small matrix → Brute force
- Only row-wise sorted → Binary search on rows
- Row-wise + column-wise sorted → Staircase search (best)

31. Find Peak Element (2D Matrix)

Problem Statement

You are given a **0-indexed** matrix `mat` of size `n × m` such that **no two adjacent cells are equal**.
A **peak element** is an element that is **strictly greater** than its adjacent neighbors:

- top `(i-1, j)`
- bottom `(i+1, j)`
- left `(i, j-1)`
- right `(i, j+1)`

The matrix is assumed to be surrounded by a boundary of `-1`.

Your task is to find **any one peak element** and return its position `[i, j]`.

> Multiple peaks may exist. Returning **any valid peak index** is acceptable.

Example 1

```
Input:
mat = [
  [5, 10, 8],
  [4, 25, 7],
  [3, 9, 6]
]

Output:
[1, 1]

Explanation:
25 is greater than its neighbors (10, 7, 4, 9)
```

Example 2

```
Input:
mat = [
  [1, 2, 3],
  [6, 5, 4],
  [7, 8, 9]
]

Output:
[2, 2]

Explanation:
9 is greater than its neighbors (8, 4)
```

Optimal Approach (Binary Search on Columns)

Core Idea

This problem extends the **1D peak element** idea to **2D**.

Instead of binary search on rows, we:

- Apply **binary search on columns**
- In each middle column, find the **maximum element**
- Compare it with its **left and right neighbors**

Why This Works

For a chosen middle column:

- Let `(row, mid)` be the **maximum element** in that column
- Since it is already the largest in its column:

- Only left and right neighbors can invalidate it as a peak

Now:

- If it is greater than both left and right → **peak found**
- If left neighbor is larger → peak exists in **left half**
- If right neighbor is larger → peak exists in **right half**

This guarantees progress just like binary search.

Steps

1. Set `low = 0`, `high = m - 1`
2. While `low <= high`:
    - `mid = (low + high) / 2`
    - Find the row index of the **maximum element** in column `mid`
    - Compare this element with its left and right neighbors
3. Decide direction:
    - Peak found → return position
    - Left larger → search left
    - Right larger → search right

Helper Function

Find the row index of the **maximum element in a given column**.

C++ Code (Optimal)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    // Returns row index of max element in a column
    int maxElement(vector<vector<int>>& mat, int col) {
        int n = mat.size();
        int idx = 0;

        for (int i = 1; i < n; i++) {
            if (mat[i][col] > mat[idx][col])
                idx = i;
        }
        return idx;
    }

    vector<int> findPeakGrid(vector<vector<int>>& mat) {
        int n = mat.size();
        int m = mat[0].size();

        int low = 0, high = m - 1;

        while (low <= high) {
            int mid = (low + high) / 2;
            int row = maxElement(mat, mid);

            int left  = (mid - 1 >= 0) ? mat[row][mid - 1] : INT_MIN;
            int right = (mid + 1 < m) ? mat[row][mid + 1] : INT_MIN;

            if (mat[row][mid] > left && mat[row][mid] > right) {
                return {row, mid};
            }
            else if (left > mat[row][mid]) {
                high = mid - 1;
            }
            else {
```

```
                low = mid + 1;
            }
        }
        return {-1, -1};
    }
};

int main() {
    vector<vector<int>> mat = {
        {4, 2, 5, 1, 4, 5},
        {2, 9, 3, 2, 3, 2},
        {1, 7, 6, 0, 1, 3},
        {3, 6, 2, 3, 7, 2}
    };

    Solution sol;
    vector<int> ans = sol.findPeakGrid(mat);
    cout << ans[0] << " " << ans[1] << endl;
}
```

Complexity Analysis

- Finding max in a column: $O(N)$
- Binary search on columns: $O(\log M)$

Time Complexity: $O(N \log M)$
Space Complexity: $O(1)$

32. Median of Row-Wise Sorted Matrix

Problem Statement

You are given a matrix of size `M × N` where **each row is sorted in non-decreasing order**. `M × N` is always **odd**.

You need to find the **median** of the matrix.

Median definition:

- Since total elements are odd, median is the **(M×N)/2-th element (0-based)** in the sorted order of all elements.

Example 1

```
Input:
1 4 9
2 5 6
3 8 7

Output:
5

Explanation:
Sorted order:
1 2 3 4 5 6 7 8 9
Median = 5
```

Example 2

```
Input:
1 3 8
2 3 4
1 2 5
```

```
Output:
3

Explanation:
Sorted order:
1 1 2 2 3 3 4 5 8
Median = 3
```

Brute Force Approach

Idea

Store all elements of the matrix into a single array, sort it, and return the middle element.

Steps

1. Create an empty array
2. Push all elements of the matrix into it
3. Sort the array
4. Return element at index `(M×N)/2`

C++ Code (Brute Force)

```cpp
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int findMedian(vector<vector<int>>& matrix) {
        vector<int> arr;
```

```
        for (auto& row : matrix) {
            for (int x : row) {
                arr.push_back(x);
            }
        }

        sort(arr.begin(), arr.end());
        return arr[arr.size() / 2];
    }
};

int main() {
    vector<vector<int>> matrix = {
        {1, 4, 9},
        {2, 5, 6},
        {3, 8, 7}
    };

    Solution obj;
    cout << obj.findMedian(matrix) << endl;
    return 0;
}
```

Complexity (Brute Force)

- **Time:** $O(M \times N \log(M \times N))$
- **Space:** $O(M \times N)$

Optimal Approach (Binary Search on Value Space)

Key Observation

- Rows are sorted, but the matrix is **not globally sorted**

- Median depends on **value**, not index
- We binary search on the **range of values**, not positions

Core Idea

Instead of sorting everything:

- Guess a value `mid`
- Count how many elements in the matrix are `≤ mid`
- If count < required → median is larger
- Else → median is smaller or equal

Search Space

- `low` = minimum element among first elements of all rows
- `high` = maximum element among last elements of all rows

Required count for median:

```
(m × n + 1) / 2
```

Counting ≤ mid

Since each row is sorted:

- Use `upper_bound` on each row
- This gives count of elements ≤ mid in that row

Steps

1. Find `low` and `high`
2. While `low < high`:
    - `mid = (low + high) / 2`
    - Count elements ≤ `mid`
3. Adjust range based on count
4. Final `low` is the median

C++ Code (Optimal)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int findMedian(vector<vector<int>>& matrix) {
        int rows = matrix.size();
        int cols = matrix[0].size();

        int low = matrix[0][0];
        int high = matrix[0][cols - 1];

        for (int i = 1; i < rows; i++) {
            low = min(low, matrix[i][0]);
            high = max(high, matrix[i][cols - 1]);
        }

        int required = (rows * cols + 1) / 2;

        while (low < high) {
```

```
            int mid = (low + high) / 2;
            int count = 0;

            for (int i = 0; i < rows; i++) {
                count += upper_bound(matrix[i].begin(),
                                     matrix[i].end(),
                                     mid) - matrix[i].begin();
            }

            if (count < required)
                low = mid + 1;
            else
                high = mid;
        }

        return low;
    }
};

int main() {
    vector<vector<int>> matrix = {
        {1, 3, 5},
        {2, 6, 9},
        {3, 6, 9}
    };

    Solution obj;
    cout << obj.findMedian(matrix) << endl;
    return 0;
}
```

Complexity (Optimal)

- **Time:** $O(M \times \log N \times \log(\text{valueRange}))$
- **Space:** $O(1)$

Comparison Summary

Approach	Time	Space
Brute Force	`O(MN log(MN))`	`O(MN)`
Optimal	`O(M log N log range)`	`O(1)`